

Dual-GPU Qwen Knowledge Graph Pipeline

Data Engineering Team

June 18, 2026

Overview

This document contains a comprehensive toolkit of scripts utilized for generating a medical Knowledge Graph from the Kurdish medical dataset. It leverages a dual-GPU deployment of the Qwen3.6:35b model via Ollama for semantic relation extraction and Neo4j for storage. Below is a summary table containing links to the code for each step in the pipeline, alongside an explanation of its purpose.

Script Name	Section Link	Purpose / Why We Use It
Data Preparation		
Split Dataset	Section 1	Splits the dataset into two parts for parallel processing on two GPUs.
Knowledge Graph Generators		
KG Generator (GPU 0)	Section 2	Extracts medical triples using Ollama on GPU 0 and stores them in Neo4j.
KG Generator (GPU 1)	Section 3	Extracts medical triples using Ollama on GPU 1 and stores them in Neo4j.
Run KG Script (GPU 0)	Section 4	Shell script to run the GPU 0 knowledge graph generator in the background.
Run KG Script (GPU 1)	Section 5	Shell script to run the GPU 1 knowledge graph generator in the background.
Stop KG Script	Section 6	Gracefully stops the background Python processes and Ollama servers.
Setup & Environment		
Setup Script	Section 7	Prepares the environment by installing dependencies and configuring Neo4j/Ollama.
Setup Runner	Section 8	Python wrapper to execute the setup shell script in the background.
Model Loader (GPU 0)	Section 9	A dedicated script to manage loading the Ollama model into GPU 0 memory.
Model Loader (GPU 1)	Section 10	A dedicated script to manage loading the Ollama model into GPU 1 memory.
Requirements	Section 11	Lists the necessary Python packages for the pipeline.

Table 1: Dual-GPU Knowledge Graph generation scripts categorized by functional domain.

1 Split Dataset

Purpose: Splits the dataset into two equal halves to allow parallel processing across the two GPUs.

Listing 1: split_dataset.py

```
1 import json
2 import math
3 import os
4
5 def split_dataset(input_path, output_part1, output_part2):
6     print(f"Reading {input_path}...")
7     with open(input_path, 'r', encoding='utf-8') as f:
8         data = json.load(f)
9
10    total_entries = len(data)
11    halfway = math.ceil(total_entries / 2)
12
13    part1 = data[:halfway]
14    part2 = data[halfway:]
15
16    print(f"Total entries: {total_entries}")
17    print(f"Part 1: {len(part1)} entries")
18    print(f"Part 2: {len(part2)} entries")
19
20    with open(output_part1, 'w', encoding='utf-8') as f:
21        json.dump(part1, f, ensure_ascii=False, indent=2)
22
23    with open(output_part2, 'w', encoding='utf-8') as f:
24        json.dump(part2, f, ensure_ascii=False, indent=2)
25
26    print(f"Successfully saved to {output_part1} and {output_part2}")
27
28 if __name__ == "__main__":
29     DATASET_PATH = "kurdish_medical_corpus_kmc.json"
30     PART1_PATH = "kurdish_medical_corpus_kmc_part1.json"
31     PART2_PATH = "kurdish_medical_corpus_kmc_part2.json"
32
33     if os.path.exists(DATASET_PATH):
34         split_dataset(DATASET_PATH, PART1_PATH, PART2_PATH)
35     else:
36         print(f"Error: {DATASET_PATH} not found.")
```

2 Knowledge Graph Generator (GPU 0)

Purpose: We use this script to extract medical triples from the first half of the Kurdish dataset using the Ollama service on GPU 0.

Listing 2: kg_generator_gpu_0.py

```
1 import json
2 import os
3 import re
4 import time
```

```

5 import logging
6 import threading
7 from concurrent.futures import ThreadPoolExecutor
8 from typing import List, Dict, Optional
9 from tqdm import tqdm
10 from ollama import Client
11 from neo4j import GraphDatabase
12 from dotenv import load_dotenv
13
14 # Load environment variables
15 load_dotenv()
16
17 # -----
18 # GPU 0 → Ollama on port 11434
19 # -----
20
21 # Setup Logging
22 logging.basicConfig(
23     level=logging.INFO,
24     format='%(asctime)s - %(levelname)s - %(message)s',
25     handlers=[
26         logging.FileHandler("kg_pipeline_gpu_0.log"),
27         logging.StreamHandler()
28     ]
29 )
30 logger = logging.getLogger(__name__)
31
32 # — Configuration —
33 OLLAMA_HOST = os.getenv("OLLAMA_HOST_GPU0", "http://localhost:11434")
34 OLLAMA_MODEL = "qwen3.6:35b"
35 NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
36 NEO4J_USER = os.getenv("NEO4J_USER", "neo4j")
37 NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD", "password")
38 DATASET_PATH = "kurdish_medical_corpus_kmc_part1.json"
39 CHECKPOINT_PATH = "processed_ids_gpu_0.txt"
40 OUTPUT_FILE = "extracted_triples_gpu_0.jsonl"
41
42 # Qwen3 generation options - generous token budgets so CoT has room to breathe
43 OLLAMA_OPTIONS = {
44     "temperature": 0.6,      # Qwen3 docs recommend 0.6 for balanced CoT
45     "top_p": 0.95,          # Qwen3 recommended value
46     "top_k": 20,            # Qwen3 recommended value
47     "min_p": 0,
48     "num_predict": 8192,    # Large budget: CoT + JSON output
49     "num_ctx": 16384,       # Context window - enough for long Kurdish texts + prompt
50     "repeat_penalty": 1.0,
51 }
52
53 # — Robust JSON extractor —
54 def _extract_json_from_text(text: str) -> Optional[list]:
55     """
56     Extract the first JSON array from arbitrary text (including CoT reasoning).
57     Qwen3 thinks in <think...></think> blocks before giving the answer;
58     we must scan the *entire* response for any valid JSON array.
59     """
60     if not text:
61         return None
62
63     # Strip think blocks if present (Qwen3 CoT)

```

```

64 text_no_think = re.sub(r'<think>.*?</think>', '', text, flags=re.DOTALL).strip()
65
66 # Prefer text after think block; fall back to full text
67 sources = [text_no_think, text] if text_no_think else [text]
68
69 for src in sources:
70     # Try to find a JSON array directly
71     for match in re.finditer(r'\[', src):
72         start = match.start()
73         depth = 0
74         for i, ch in enumerate(src[start:], start):
75             if ch == '[':
76                 depth += 1
77             elif ch == ']':
78                 depth -= 1
79                 if depth == 0:
80                     candidate = src[start:i + 1]
81                     try:
82                         return json.loads(candidate)
83                     except json.JSONDecodeError:
84                         break
85
86     # Try wrapped in code block: ```json ... ```
87     cb = re.search(r'```(?:json)?\s*(\[.*?\])\s*```', src, re.DOTALL)
88     if cb:
89         try:
90             return json.loads(cb.group(1))
91         except json.JSONDecodeError:
92             pass
93
94     # Try a bare JSON object that IS a list item (dict)
95     for match in re.finditer(r'\{', src):
96         start = match.start()
97         depth = 0
98         for i, ch in enumerate(src[start:], start):
99             if ch == '{':
100                 depth += 1
101             elif ch == '}':
102                 depth -= 1
103                 if depth == 0:
104                     candidate = src[start:i + 1]
105                     try:
106                         obj = json.loads(candidate)
107                         if isinstance(obj, dict):
108                             return [obj]
109                     except json.JSONDecodeError:
110                         break
111
112     return None
113
114
115 # — Validation —————
116 VALID_TYPES = {
117     "Disease", "Symptom", "Treatment", "Medication",
118     "Anatomy", "Provider", "Organization"
119 }
120
121 def _validate_triples(raw_triples: list) -> List[Dict]:
122     """Strict quality gate: every triple must have non-empty head/relation/tail."""

```

```

123     valid = []
124     for t in raw_triples:
125         if not isinstance(t, dict):
126             continue
127         head = str(t.get('head', '')).strip()
128         relation = str(t.get('relation', '')).strip()
129         tail = str(t.get('tail', '')).strip()
130         head_type = str(t.get('head_type', 'Entity')).strip()
131         tail_type = str(t.get('tail_type', 'Entity')).strip()
132
133         if len(head) < 2 or len(tail) < 2 or len(relation) < 2:
134             continue
135
136         # Normalise types
137         if head_type not in VALID_TYPES:
138             head_type = 'Entity'
139         if tail_type not in VALID_TYPES:
140             tail_type = 'Entity'
141
142         valid.append({
143             "head": head,
144             "head_type": head_type,
145             "relation": relation,
146             "tail": tail,
147             "tail_type": tail_type,
148         })
149     return valid
150
151
152 # — Main class —————
153 class KMCKnowledgeGraph:
154     def __init__(self):
155         logger.info(f"Connecting to Ollama at {OLLAMA_HOST} (GPU 0) ...")
156         self.client = Client(host=OLLAMA_HOST)
157
158         # Warm-up: ensure model is loaded on GPU 0
159         try:
160             logger.info(f"Pre-loading model {OLLAMA_MODEL} on GPU 0 ...")
161             self.client.generate(
162                 model=OLLAMA_MODEL,
163                 prompt="Hello",
164                 options={"num_predict": 1}
165             )
166             logger.info("Model pre-loaded successfully.")
167         except Exception as e:
168             logger.warning(f"Model pre-load warning (continuing): {e}")
169
170         try:
171             self.driver = GraphDatabase.driver(
172                 NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD)
173             )
174             self.driver.verify_connectivity()
175             logger.info("Connected to Neo4j successfully.")
176         except Exception as e:
177             logger.error(f"Failed to connect to Neo4j: {e}")
178             raise
179
180         self.processed_ids = self._load_checkpoints()
181         self.lock = threading.RLock()

```

```

182
183     # Persistent file handles for faster I/O
184     self.checkpoint_file = open(CHECKPOINT_PATH, 'a', encoding='utf-8')
185     self.output_file = open(OUTPUT_FILE, 'a', encoding='utf-8')
186     self.neo4j_ok = True
187
188     # — Checkpointing —————
189     def _load_checkpoints(self) -> set:
190         if os.path.exists(CHECKPOINT_PATH):
191             with open(CHECKPOINT_PATH, 'r', encoding='utf-8') as f:
192                 return set(line.strip() for line in f if line.strip())
193         return set()
194
195     def _save_checkpoint(self, entry_id: str):
196         with self.lock:
197             self.checkpoint_file.write(f"{entry_id}\n")
198             self.checkpoint_file.flush()
199             self.processed_ids.add(entry_id)
200
201     def close(self):
202         try:
203             self.driver.close()
204         except:
205             pass
206         try:
207             self.checkpoint_file.close()
208             self.output_file.close()
209         except:
210             pass
211
212     # — Text cleaning —————
213     def clean_text(self, text: str) -> str:
214         if not text:
215             return ""
216         text = re.sub(r'\s+', ' ', text)
217         return text.strip()
218
219     # — Triple extraction —————
220     def extract_triples(self, text: str, max_retries: int = 5) -> List[Dict]:
221         """
222         Extract knowledge-graph triples from Kurdish medical text.
223
224         KEY DESIGN DECISION for Qwen3.6:35b
225
226         • We do NOT pass format="json" to Ollama.
227           Qwen3 uses internal Chain-of-Thought (<think...></think>) before
228           writing the final answer. Forcing JSON mode suppresses that CoT
229           block which drastically hurts extraction quality and often produces
230           empty output.
231         • Instead we prompt the model to wrap its final JSON in a clear
232           delimiter (===JSON_START=== / ===JSON_END===) so our parser can
233           find it reliably even inside a long reasoning trace.
234         • num_predict is set to 8192 to give the CoT enough room.
235         """
236         prompt = (
237             "You are an elite Kurdish medical linguist and knowledge graph engineer.\n
238
239             "Your task is to extract high-quality, semantically rich medical triples "
240             "from the provided Kurdish text.\n\n"

```

```

240
241 "CRITICAL QUALITY REQUIREMENTS:\n"
242 "1. LANGUAGE: All medical entities and relationships MUST be in natural, "
243 "accurate Kurdish (Sorani/Central Kurdish).\n"
244 "2. MEANINGFUL RELATIONS: Use precise medical verbs, never 'has' or 'is'.
n"
245 "   Examples:\n"
246 "   - هۆی 'دهبۆتە' (causes)\n"
247 "   - بۆ 'نیشانهی' (is a symptom of)\n"
248 "   - 'بەکاردهی' 'چارەسەر' (used for treatment of)\n"
249 "   - 'سەر' 'کاردهکاتە' (affects)\n"
250 "   - 'له' 'بەشێکە' (is part of)\n"
251 "3. ENTITY TYPES: head_type and tail_type MUST be ONE of: "
252 "[Disease, Symptom, Treatment, Medication, Anatomy, Provider, Organization
].\n"
253 "4. DATA INTEGRITY: Every triple MUST have a valid head, relation, and
tail. "
254 "No empty strings.\n"
255 "5. COMPLETENESS: Extract ALL meaningful medical facts from the text. "
256 "Aim for at least 3-8 triples per passage if the text contains medical
info.\n\n"
257
258 f"Kurdish Text:\n{text}\n\n"
259
260 "Think carefully about the medical entities and their relationships.\n"
261 "After your reasoning, output YOUR FINAL ANSWER between these exact
markers:\n"
262 "===JSON_START===\n"
263 "[\n"
264 "  {\n"
265 "    \"head\": \"Entity Name in Kurdish\", \"\n"
266 "    \"head_type\": \"English Label\", \"\n"
267 "    \"relation\": \"Detailed Kurdish Relationship\", \"\n"
268 "    \"tail\": \"Target Entity in Kurdish\", \"\n"
269 "    \"tail_type\": \"English Label\" \"\n"
270 "  } \"\n"
271 "]" \"\n"
272 "===JSON_END===\n\n"
273 "If the text contains NO meaningful medical relationships, output:\n"
274 "===JSON_START===\n"
275 "[] \"\n"
276 "===JSON_END===\n"
277 )
278
279 for attempt in range(max_retries):
280     try:
281         response = self.client.generate(
282             model=OLLAMA_MODEL,
283             prompt=prompt,
284             # ← NO format="json" – allow full CoT output
285             options=OLLAMA_OPTIONS,
286         )
287
288         # Resolve response object
289         if isinstance(response, dict):
290             content = response.get('response', '')
291         else:
292             content = getattr(response, 'response', '')
293

```

```

294         if not content or not content.strip():
295             logger.warning(f"[GPU0] Attempt {attempt+1}: Empty response.
...Retrying")
296             time.sleep(1)
297             continue
298
299         logger.debug(f"[GPU0] Raw response length: {len(content)} chars")
300
301         # — Priority 1: extract between our explicit markers —
302         marker_match = re.search(
303             r'===JSON_START===\s*(.*?)\s*===JSON_END===',
304             content, re.DOTALL
305         )
306         if marker_match:
307             json_str = marker_match.group(1).strip()
308             try:
309                 data = json.loads(json_str)
310                 if isinstance(data, list):
311                     raw_triples = [i for i in data if isinstance(i, dict)]
312                     valid = _validate_triples(raw_triples)
313                     if valid:
314                         logger.info(f"[GPU0] Attempt {attempt+1}: Extracted {
len(valid)} valid triples.")
315                         return valid
316                     elif not raw_triples:
317                         # Model explicitly said no triples
318                         logger.info(f"[GPU0] Model found no medical
relationships in this text.")
319                         return []
320                     # else: had triples but all failed validation → retry
321                     logger.warning(f"[GPU0] Attempt {attempt+1}: All triples
failed validation. ...Retrying")
322                     continue
323             except json.JSONDecodeError:
324                 pass # Fall through to robust extractor
325
326         # — Priority 2: robust scan of full response —
327         raw_list = _extract_json_from_text(content)
328         if raw_list is not None:
329             if isinstance(raw_list, list):
330                 if not raw_list:
331                     logger.info(f"[GPU0] Attempt {attempt+1}: Model returned
empty list.")
332                     return []
333                     valid = _validate_triples(raw_list)
334                     if valid:
335                         logger.info(f"[GPU0] Attempt {attempt+1}: Extracted {len(
valid)} valid triples (fallback parser).")
336                         return valid
337                         logger.warning(f"[GPU0] Attempt {attempt+1}: Fallback triples
failed validation. ...Retrying")
338                         continue
339
340             logger.warning(f"[GPU0] Attempt {attempt+1}: No JSON found in response
. ...Retrying")
341             time.sleep(1)
342
343         except Exception as e:
344             logger.warning(f"[GPU0] Attempt {attempt+1} Exception: {e}")

```

```

345         time.sleep(2)
346
347         logger.error("[GPU0] All retries exhausted. Returning empty list.")
348         return []
349
350     # — Neo4j storage —————
351     def store_triples(self, triples: List[Dict]):
352         """Store multiple triples in a single Neo4j session for efficiency."""
353         if not triples or not self.neo4j_ok:
354             return
355
356         try:
357             with self.driver.session() as session:
358                 for triple in triples:
359                     head = triple.get('head')
360                     h_type = triple.get('head_type', 'Entity')
361                     rel = triple.get('relation', 'RELATED_TO')
362                     tail = triple.get('tail')
363                     t_type = triple.get('tail_type', 'Entity')
364
365                     if not head or not tail:
366                         continue
367
368                     # Neo4j label must be alphanumeric
369                     h_label = re.sub(r'^a-zA-Z0-9_', '', str(h_type)) or "Entity"
370                     if h_label[0].isdigit(): h_label = "L_" + h_label
371
372                     t_label = re.sub(r'^a-zA-Z0-9_', '', str(t_type)) or "Entity"
373                     if t_label[0].isdigit(): t_label = "L_" + t_label
374
375                     # Sanitize relation
376                     rel_clean = re.sub(r'^a-zA-Z0-9_', '_', str(rel)).upper()
377                     rel_clean = re.sub(r'_+', '_', rel_clean).strip('_') or "
RELATED_TO"
378
379                     if rel_clean[0].isdigit(): rel_clean = "R_" + rel_clean
380
381                     query = (
382                         f"MERGE (h:{h_label} {{name: $head}}) "
383                         f"MERGE (t:{t_label} {{name: $tail}}) "
384                         f"MERGE (h)-[r:{rel_clean}]->(t) "
385                     )
386                     session.run(query, head=head, tail=tail)
387         except Exception as e:
388             logger.error(f"[GPU0] Neo4j store error: {e}")
389             self.neo4j_ok = False
390
391     # — Dataset processing —————
392     def process_entry(self, entry: Dict):
393         """Process a single entry from the dataset."""
394         entry_id = str(entry.get('id', '')).strip()
395
396         with self.lock:
397             if not entry_id or entry_id in self.processed_ids:
398                 return
399
400         raw_text = entry.get('response', '')
401         if not raw_text or not raw_text.strip():
402             self._save_checkpoint(entry_id)
403             return

```

```

403
404     cleaned_text = self.clean_text(raw_text)
405     triples = self.extract_triples(cleaned_text)
406
407     if triples:
408         # — Persist to JSONL —
409         try:
410             record = {"id": entry_id, "triples": triples}
411             line = json.dumps(record, ensure_ascii=False) + "\n"
412             with self.lock:
413                 self.output_file.write(line)
414                 self.output_file.flush()
415             logger.info(f"[GPU0] ID={entry_id}: Extracted {len(triples)} triples."
416         )
417         except Exception as e:
418             logger.error(f"[GPU0] Failed to write to {OUTPUT_FILE}: {e}")
419
420         # — Persist to Neo4j —
421         self.store_triples(triples)
422
423     self._save_checkpoint(entry_id)
424
425 def process_dataset(self):
426     logger.info(f"[GPU0] Loading dataset from {DATASET_PATH} ...")
427     try:
428         with open(DATASET_PATH, 'r', encoding='utf-8') as f:
429             data = json.load(f)
430     except Exception as e:
431         logger.error(f"[GPU0] Could not load dataset: {e}")
432         return
433
434     total = len(data)
435     with self.lock:
436         already = len(self.processed_ids)
437
438     remaining_data = [e for e in data if str(e.get('id', '')).strip() not in self.
439 processed_ids]
440     logger.info(f"[GPU0] {total} total | {already} done | {len(remaining_data)}
441 remaining.")
442
443     if not remaining_data:
444         logger.info("[GPU0] No entries to process.")
445         return
446
447     # Use ThreadPoolExecutor for concurrent requests
448     # max_workers=3 is a safe balance for GPU 35b models to overlap I/O and
449 compute
450     MAX_WORKERS = 3
451     with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
452         list(tqdm(
453             executor.map(self.process_entry, remaining_data),
454             total=len(remaining_data),
455             desc="GPU0"
456         ))
457
458     logger.info("[GPU0] Pipeline finished successfully.")
459
460 if __name__ == "__main__":

```

```

458     kg = KMCKnowledgeGraph()
459     try:
460         kg.process_dataset()
461     except KeyboardInterrupt:
462         logger.info("[GPU0] Process interrupted by user. Progress saved.")
463     except Exception as e:
464         logger.error(f"[GPU0] Fatal error: {e}")
465     finally:
466         kg.close()

```

3 Knowledge Graph Generator (GPU 1)

Purpose: We use this script to extract medical triples from the second half of the Kurdish dataset using the Ollama service on GPU 1.

Listing 3: kg_generator_gpu_1.py

```

1  import json
2  import os
3  import re
4  import time
5  import logging
6  import threading
7  from concurrent.futures import ThreadPoolExecutor
8  from typing import List, Dict, Optional
9  from tqdm import tqdm
10 from ollama import Client
11 from neo4j import GraphDatabase
12 from dotenv import load_dotenv
13
14 # Load environment variables
15 load_dotenv()
16
17 # -----
18 # GPU 1 → Ollama on port 11435
19 # -----
20
21 # Setup Logging
22 logging.basicConfig(
23     level=logging.INFO,
24     format='%(asctime)s - %(levelname)s - %(message)s',
25     handlers=[
26         logging.FileHandler("kg_pipeline_gpu_1.log"), # ← correct log file
27         logging.StreamHandler()
28     ]
29 )
30 logger = logging.getLogger(__name__)
31
32 # — Configuration —
33 OLLAMA_HOST = os.getenv("OLLAMA_HOST_GPU1", "http://localhost:11435") # ← GPU 1
34 port
35 OLLAMA_MODEL = "qwen3.6:35b"
36 NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
37 NEO4J_USER = os.getenv("NEO4J_USER", "neo4j")
38 NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD", "password")

```

```

38 DATASET_PATH = "kurdish_medical_corpus_kmc_part2.json"      # ← GPU 1 dataset half
39 CHECKPOINT_PATH = "processed_ids_gpu_1.txt"                 # ← GPU 1 checkpoint
40 OUTPUT_FILE   = "extracted_triples_gpu_1.jsonl"             # ← GPU 1 output
41
42 # Qwen3 generation options - generous token budgets so CoT has room to breathe
43 OLLAMA_OPTIONS = {
44     "temperature": 0.6,      # Qwen3 docs recommend 0.6 for balanced CoT
45     "top_p": 0.95,           # Qwen3 recommended value
46     "top_k": 20,             # Qwen3 recommended value
47     "min_p": 0,
48     "num_predict": 8192,     # Large budget: CoT + JSON output
49     "num_ctx": 16384,        # Context window - enough for long Kurdish texts + prompt
50     "repeat_penalty": 1.0,
51 }
52
53 # — Robust JSON extractor —————
54 def _extract_json_from_text(text: str) -> Optional[list]:
55     """
56     Extract the first JSON array from arbitrary text (including CoT reasoning).
57     Qwen3 thinks in <think...></think> blocks before giving the answer;
58     we must scan the *entire* response for any valid JSON array.
59     """
60     if not text:
61         return None
62
63     # Strip think blocks if present (Qwen3 CoT)
64     text_no_think = re.sub(r'<think>.*?</think>', '', text, flags=re.DOTALL).strip()
65
66     # Prefer text after think block; fall back to full text
67     sources = [text_no_think, text] if text_no_think else [text]
68
69     for src in sources:
70         # Try to find a JSON array directly
71         for match in re.finditer(r'\[', src):
72             start = match.start()
73             depth = 0
74             for i, ch in enumerate(src[start:], start):
75                 if ch == '[':
76                     depth += 1
77                 elif ch == ']':
78                     depth -= 1
79                 if depth == 0:
80                     candidate = src[start:i + 1]
81                     try:
82                         return json.loads(candidate)
83                     except json.JSONDecodeError:
84                         break
85
86     # Try wrapped in code block: ```json ... ```
87     cb = re.search(r'```(?:json)?\s*(\[.*?\])\s*```', src, re.DOTALL)
88     if cb:
89         try:
90             return json.loads(cb.group(1))
91         except json.JSONDecodeError:
92             pass
93
94     # Try a bare JSON object that IS a list item (dict)
95     for match in re.finditer(r'\{', src):
96         start = match.start()

```

```

97         depth = 0
98         for i, ch in enumerate(src[start:], start):
99             if ch == '{':
100                 depth += 1
101             elif ch == '}':
102                 depth -= 1
103                 if depth == 0:
104                     candidate = src[start:i + 1]
105                     try:
106                         obj = json.loads(candidate)
107                         if isinstance(obj, dict):
108                             return [obj]
109                     except json.JSONDecodeError:
110                         break
111
112     return None
113
114
115 # — Validation —————
116 VALID_TYPES = {
117     "Disease", "Symptom", "Treatment", "Medication",
118     "Anatomy", "Provider", "Organization"
119 }
120
121 def _validate_triples(raw_triples: list) -> List[Dict]:
122     """Strict quality gate: every triple must have non-empty head/relation/tail."""
123     valid = []
124     for t in raw_triples:
125         if not isinstance(t, dict):
126             continue
127         head = str(t.get('head', '')).strip()
128         relation = str(t.get('relation', '')).strip()
129         tail = str(t.get('tail', '')).strip()
130         head_type = str(t.get('head_type', 'Entity')).strip()
131         tail_type = str(t.get('tail_type', 'Entity')).strip()
132
133         if len(head) < 2 or len(tail) < 2 or len(relation) < 2:
134             continue
135
136         # Normalise types
137         if head_type not in VALID_TYPES:
138             head_type = 'Entity'
139         if tail_type not in VALID_TYPES:
140             tail_type = 'Entity'
141
142         valid.append({
143             "head": head,
144             "head_type": head_type,
145             "relation": relation,
146             "tail": tail,
147             "tail_type": tail_type,
148         })
149     return valid
150
151
152 # — Main class —————
153 class KMCKnowledgeGraph:
154     def __init__(self):
155         logger.info(f"Connecting to Ollama at {OLLAMA_HOST} (GPU 1) ...")

```

```

156     self.client = Client(host=OLLAMA_HOST)
157
158     # Warm-up: ensure model is loaded on GPU 1
159     try:
160         logger.info(f"Pre-loading model {OLLAMA_MODEL} on GPU 1 ...")
161         self.client.generate(
162             model=OLLAMA_MODEL,
163             prompt="Hello",
164             options={"num_predict": 1}
165         )
166         logger.info("Model pre-loaded successfully.")
167     except Exception as e:
168         logger.warning(f"Model pre-load warning (continuing): {e}")
169
170     try:
171         self.driver = GraphDatabase.driver(
172             NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD)
173         )
174         self.driver.verify_connectivity()
175         logger.info("Connected to Neo4j successfully.")
176     except Exception as e:
177         logger.error(f"Failed to connect to Neo4j: {e}")
178         raise
179
180     self.processed_ids = self._load_checkpoints()
181     self.lock = threading.RLock()
182
183     # Persistent file handles for faster I/O
184     self.checkpoint_file = open(CHECKPOINT_PATH, 'a', encoding='utf-8')
185     self.output_file = open(OUTPUT_FILE, 'a', encoding='utf-8')
186     self.neo4j_ok = True
187
188     # — Checkpointing —————
189     def _load_checkpoints(self) -> set:
190         if os.path.exists(CHECKPOINT_PATH):
191             with open(CHECKPOINT_PATH, 'r', encoding='utf-8') as f:
192                 return set(line.strip() for line in f if line.strip())
193         return set()
194
195     def _save_checkpoint(self, entry_id: str):
196         with self.lock:
197             self.checkpoint_file.write(f"{entry_id}\n")
198             self.checkpoint_file.flush()
199             self.processed_ids.add(entry_id)
200
201     def close(self):
202         try:
203             self.driver.close()
204         except:
205             pass
206         try:
207             self.checkpoint_file.close()
208             self.output_file.close()
209         except:
210             pass
211
212     # — Text cleaning —————
213     def clean_text(self, text: str) -> str:
214         if not text:

```

```

215         return ""
216     text = re.sub(r'\s+', ' ', text)
217     return text.strip()
218
219 # — Triple extraction —————
220 def extract_triples(self, text: str, max_retries: int = 5) -> List[Dict]:
221     """
222     Extract knowledge-graph triples from Kurdish medical text.
223
224     KEY DESIGN DECISION for Qwen3.6:35b
225     """
226     • We do NOT pass format="json" to Ollama.
227       Qwen3 uses internal Chain-of-Thought (<think...></think>) before
228       writing the final answer. Forcing JSON mode suppresses that CoT
229       block which drastically hurts extraction quality and often produces
230       empty output.
231     • Instead we prompt the model to wrap its final JSON in a clear
232       delimiter (===JSON_START=== / ===JSON_END===) so our parser can
233       find it reliably even inside a long reasoning trace.
234     • num_predict is set to 8192 to give the CoT enough room.
235     """
236     prompt = (
237         "You are an elite Kurdish medical linguist and knowledge graph engineer.\n
238
239         "Your task is to extract high-quality, semantically rich medical triples "
240         "from the provided Kurdish text.\n\n"
241
242         "CRITICAL QUALITY REQUIREMENTS:\n"
243         "1. LANGUAGE: All medical entities and relationships MUST be in natural, "
244         "accurate Kurdish (Sorani/Central Kurdish).\n"
245         "2. MEANINGFUL RELATIONS: Use precise medical verbs, never 'has' or 'is'.\n
246     n"
247         "    Examples:\n"
248         "    - هۆی 'دهبۆتە' (causes)\n"
249         "    - بۆ 'نیشانیە' (is a symptom of)\n"
250         "    - چارهسەری بۆ 'بەکاردێت' (used for treatment of)\n"
251         "    - سەر 'کاردهکاته' (affects)\n"
252         "    - لێ 'بەشێکە' (is part of)\n"
253         "3. ENTITY TYPES: head_type and tail_type MUST be ONE of: "
254         "[Disease, Symptom, Treatment, Medication, Anatomy, Provider, Organization
255     ].\n"
256         "4. DATA INTEGRITY: Every triple MUST have a valid head, relation, and
257     tail. "
258         "No empty strings.\n"
259         "5. COMPLETENESS: Extract ALL meaningful medical facts from the text. "
260         "Aim for at least 3-8 triples per passage if the text contains medical
261     info.\n\n"
262         f"Kurdish Text:\n{text}\n\n"
263
264         "Think carefully about the medical entities and their relationships.\n"
265         "After your reasoning, output YOUR FINAL ANSWER between these exact
266     markers:\n"
267         "===JSON_START===\n"
268         "[\n"
269         "    {\n"
270         "        \"head\": \"Entity Name in Kurdish\", \"n\"
271         "        \"head_type\": \"English Label\", \"n\"
272         "        \"relation\": \"Detailed Kurdish Relationship\", \"n\"

```

```

268         '        "tail": "Target Entity in Kurdish",\n'
269         '        "tail_type": "English Label"\n'
270         '    }\n'
271     "]\n"
272     "===JSON_END===\n\n"
273     "If the text contains NO meaningful medical relationships, output:\n"
274     "===JSON_START===\n"
275     "[]\n"
276     "===JSON_END==="
277 )
278
279 for attempt in range(max_retries):
280     try:
281         response = self.client.generate(
282             model=OLLAMA_MODEL,
283             prompt=prompt,
284             # ← NO format="json" — allow full CoT output
285             options=OLLAMA_OPTIONS,
286         )
287
288         # Resolve response object
289         if isinstance(response, dict):
290             content = response.get('response', '')
291         else:
292             content = getattr(response, 'response', '')
293
294         if not content or not content.strip():
295             logger.warning(f"[GPU1] Attempt {attempt+1}: Empty response.
...Retrying")
296             time.sleep(1)
297             continue
298
299         logger.debug(f"[GPU1] Raw response length: {len(content)} chars")
300
301         # — Priority 1: extract between our explicit markers —
302         marker_match = re.search(
303             r'===JSON_START===\s*(.*?)\s*===JSON_END===',
304             content, re.DOTALL
305         )
306         if marker_match:
307             json_str = marker_match.group(1).strip()
308             try:
309                 data = json.loads(json_str)
310                 if isinstance(data, list):
311                     raw_triples = [i for i in data if isinstance(i, dict)]
312                     valid = _validate_triples(raw_triples)
313                     if valid:
314                         logger.info(f"[GPU1] Attempt {attempt+1}: Extracted {
len(valid)} valid triples.")
315                         return valid
316                     elif not raw_triples:
317                         # Model explicitly said no triples
318                         logger.info(f"[GPU1] Model found no medical
relationships in this text.")
319                         return []
320                     # else: had triples but all failed validation → retry
321                     logger.warning(f"[GPU1] Attempt {attempt+1}: All triples
failed validation. ...Retrying")
322                     continue

```

```

323         except json.JSONDecodeError:
324             pass # Fall through to robust extractor
325
326         # — Priority 2: robust scan of full response —
327         raw_list = _extract_json_from_text(content)
328         if raw_list is not None:
329             if isinstance(raw_list, list):
330                 if not raw_list:
331                     logger.info(f"[GPU1] Attempt {attempt+1}: Model returned
empty list.")
332                     return []
333                 valid = _validate_triples(raw_list)
334                 if valid:
335                     logger.info(f"[GPU1] Attempt {attempt+1}: Extracted {len(
valid)} valid triples (fallback parser).")
336                     return valid
337                     logger.warning(f"[GPU1] Attempt {attempt+1}: Fallback triples
failed validation. ...Retrying")
338                     continue
339
340                 logger.warning(f"[GPU1] Attempt {attempt+1}: No JSON found in response
. ...Retrying")
341                 time.sleep(1)
342
343         except Exception as e:
344             logger.warning(f"[GPU1] Attempt {attempt+1} Exception: {e}")
345             time.sleep(2)
346
347         logger.error("[GPU1] All retries exhausted. Returning empty list.")
348         return []
349
350     # — Neo4j storage —————
351     def store_triples(self, triples: List[Dict]):
352         """Store multiple triples in a single Neo4j session for efficiency."""
353         if not triples or not self.neo4j_ok:
354             return
355
356         try:
357             with self.driver.session() as session:
358                 for triple in triples:
359                     head = triple.get('head')
360                     h_type = triple.get('head_type', 'Entity')
361                     rel = triple.get('relation', 'RELATED_TO')
362                     tail = triple.get('tail')
363                     t_type = triple.get('tail_type', 'Entity')
364
365                     if not head or not tail:
366                         continue
367
368                     # Neo4j label must be alphanumeric
369                     h_label = re.sub(r'^a-zA-Z0-9_', '', str(h_type)) or "Entity"
370                     if h_label[0].isdigit(): h_label = "L_" + h_label
371
372                     t_label = re.sub(r'^a-zA-Z0-9_', '', str(t_type)) or "Entity"
373                     if t_label[0].isdigit(): t_label = "L_" + t_label
374
375                     # Sanitize relation
376                     rel_clean = re.sub(r'^a-zA-Z0-9_', '_', str(rel)).upper()
377                     rel_clean = re.sub(r'_+', '_', rel_clean).strip('_') or "

```

```

378 RELATED_TO"
379         if rel_clean[0].isdigit(): rel_clean = "R_" + rel_clean
380
381         query = (
382             f"MERGE (h:{h_label} {{name: $head}}) "
383             f"MERGE (t:{t_label} {{name: $tail}}) "
384             f"MERGE (h)-[r:{rel_clean}]->(t) "
385         )
386         session.run(query, head=head, tail=tail)
387     except Exception as e:
388         logger.error(f"[GPU1] Neo4j store error: {e}")
389         self.neo4j_ok = False
390
391 # — Dataset processing —————
392 def process_entry(self, entry: Dict):
393     """Process a single entry from the dataset."""
394     entry_id = str(entry.get('id', '')).strip()
395
396     with self.lock:
397         if not entry_id or entry_id in self.processed_ids:
398             return
399
400     raw_text = entry.get('response', '')
401     if not raw_text or not raw_text.strip():
402         self._save_checkpoint(entry_id)
403         return
404
405     cleaned_text = self.clean_text(raw_text)
406     triples = self.extract_triples(cleaned_text)
407
408     if triples:
409         # — Persist to JSONL —
410         try:
411             record = {"id": entry_id, "triples": triples}
412             line = json.dumps(record, ensure_ascii=False) + "\n"
413             with self.lock:
414                 self.output_file.write(line)
415                 self.output_file.flush()
416             logger.info(f"[GPU1] ID={entry_id}: Extracted {len(triples)} triples."
417         )
418     except Exception as e:
419         logger.error(f"[GPU1] Failed to write to {OUTPUT_FILE}: {e}")
420
421     # — Persist to Neo4j —
422     self.store_triples(triples)
423
424     self._save_checkpoint(entry_id)
425
426 def process_dataset(self):
427     logger.info(f"[GPU1] Loading dataset from {DATASET_PATH} ...")
428     try:
429         with open(DATASET_PATH, 'r', encoding='utf-8') as f:
430             data = json.load(f)
431     except Exception as e:
432         logger.error(f"[GPU1] Could not load dataset: {e}")
433         return
434
435     total = len(data)
436     with self.lock:

```

```

435         already = len(self.processed_ids)
436
437         remaining_data = [e for e in data if str(e.get('id', '')).strip() not in self.
processed_ids]
438         logger.info(f"[GPU1] {total} total | {already} done | {len(remaining_data)}
remaining.")
439
440         if not remaining_data:
441             logger.info("[GPU1] No entries to process.")
442             return
443
444         # Use ThreadPoolExecutor for concurrent requests
445         # max_workers=3 is a safe balance for GPU 35b models to overlap I/O and
compute
446         MAX_WORKERS = 3
447         with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
448             list(tqdm(
449                 executor.map(self.process_entry, remaining_data),
450                 total=len(remaining_data),
451                 desc="GPU1"
452             ))
453
454         logger.info("[GPU1] Pipeline finished successfully.")
455
456 if __name__ == "__main__":
457     kg = KMCKnowledgeGraph()
458     try:
459         kg.process_dataset()
460     except KeyboardInterrupt:
461         logger.info("[GPU1] Process interrupted by user. Progress saved.")
462     except Exception as e:
463         logger.error(f"[GPU1] Fatal error: {e}")
464     finally:
465         kg.close()
466

```

4 Run KG Script (GPU 0)

Purpose: A shell script to execute the GPU 0 knowledge graph generator in the background.

Listing 4: run_kg_gpu_0.sh

```

1  #!/bin/bash
2
3  # Check if virtual environment exists and activate it
4  if [ -d ".venv" ]; then
5      echo "Activating virtual environment..."
6      source .venv/bin/activate
7  elif [ -d "venv" ]; then
8      echo "Activating virtual environment (venv)..."
9      source venv/bin/activate
10 fi
11
12 # Run the kg_generator_gpu_0.py in the background
13 # We use -u for unbuffered output to ensure logs appear in real-time

```

```

14 export PYTHONUNBUFFERED=1
15 nohup python3 -u kg_generator_gpu_0.py > kg_console_gpu_0.log 2>&1 &
16
17 PID=$!
18 echo "-----"
19 echo "KG Generator (GPU 0) started in background."
20 echo "PID: $PID"
21 echo "Console output: kg_console_gpu_0.log"
22 echo "To stop the process, run: kill $PID"
23 echo "-----"

```

5 Run KG Script (GPU 1)

Purpose: A shell script to execute the GPU 1 knowledge graph generator in the background.

Listing 5: run_kg_gpu_1.sh

```

1  #!/bin/bash
2
3  # Check if virtual environment exists and activate it
4  if [ -d ".venv" ]; then
5      echo "Activating virtual environment..."
6      source .venv/bin/activate
7  elif [ -d "venv" ]; then
8      echo "Activating virtual environment (venv)..."
9      source venv/bin/activate
10 fi
11
12 # Run the kg_generator_gpu_1.py in the background
13 # We use -u for unbuffered output to ensure logs appear in real-time
14 export PYTHONUNBUFFERED=1
15 nohup python3 -u kg_generator_gpu_1.py > kg_console_gpu_1.log 2>&1 &
16
17 PID=$!
18 echo "-----"
19 echo "KG Generator (GPU 1) started in background."
20 echo "PID: $PID"
21 echo "Console output: kg_console_gpu_1.log"
22 echo "To stop the process, run: kill $PID"
23 echo "-----"

```

6 Stop KG Script

Purpose: Gracefully stops the background Python generator processes and terminates any stray Ollama server instances.

Listing 6: stop_kg.sh

```

1  #!/bin/bash
2
3  # =====

```

```

4 # KMC Knowledge Graph Pipeline Shutdown Script
5 # =====
6
7 echo "-----"
8 echo "[*] Initiating shutdown of KG Pipeline and Models..."
9 echo "-----"
10
11 # 1. Stop Python Generators
12 stop_python_process() {
13     local script_name=$1
14     local gpu_id=$2
15     local pid=$(pgrep -f "python3.*$script_name")
16
17     if [ -n "$pid" ]; then
18         echo "[*] GPU $gpu_id: Stopping $script_name (PID: $pid)..."
19         kill $pid
20         sleep 2
21         # Force kill if still running
22         if ps -p $pid > /dev/null; then
23             echo "[!] GPU $gpu_id: Process persistent. Force killing..."
24             kill -9 $pid
25         fi
26     else
27         echo "[-] GPU $gpu_id: No generator process found for $script_name."
28     fi
29 }
30
31 stop_python_process "kg_generator_gpu_0.py" 0
32 stop_python_process "kg_generator_gpu_1.py" 1
33
34 # 2. Stop Ollama Instances
35 echo "[*] Cleaning up Ollama server and model instances..."
36 if pgrep -f "ollama" > /dev/null; then
37     echo "[!] Found active Ollama processes. Terminating..."
38     pkill -f "ollama"
39     sleep 3
40     # Check again and force kill if necessary
41     if pgrep -f "ollama" > /dev/null; then
42         echo "[!] Some Ollama processes are stuck. Force killing all..."
43         pkill -9 -f "ollama"
44     fi
45     echo "[+] Ollama processes terminated."
46 else
47     echo "[-] No Ollama processes found."
48 fi
49
50 echo "-----"
51 echo "[*] Shutdown complete."

```

7 Setup Script

Purpose: We use this shell script to prepare the environment, install required Python dependencies from `requirements.txt`, initialize a Neo4j Docker container, and verify the Ollama installation and model availability.

Listing 7: setup.sh

```

1  #!/bin/bash
2
3  # KMC Knowledge Graph Setup Script
4
5  echo "🔍 Starting setup for KMC Knowledge Graph Pipeline..."
6
7  # 1. Check for Python
8  if ! command -v python3 &> /dev/null; then
9      echo "🔍 Python3 is not installed. Please install it first."
10     exit 1
11 fi
12
13 # 2. Install Requirements
14 echo "🔍 Installing dependencies from requirements.txt..."
15 pip install --upgrade pip
16 pip install -r requirements.txt
17
18 # 3. Docker and Neo4j Setup
19 echo "🔍 Checking Docker status..."
20 if ! command -v docker &> /dev/null; then
21     echo "🔍 Docker not found. Installing Docker..."
22     curl -fsSL https://get.docker.com -o get-docker.sh
23     sudo sh get-docker.sh
24     sudo usermod -aG docker $USER
25     rm get-docker.sh
26     DOCKER_CMD="sudo docker"
27     echo "🔍 Docker installed. Using 'sudo docker' for this session."
28 else
29     DOCKER_CMD="docker"
30 fi
31
32 echo "🔍 Setting up Neo4j container..."
33 if ! $DOCKER_CMD ps -a --format '{{.Names}}' | grep -q "^neo4j-kmc$"; then
34     $DOCKER_CMD run -d \
35         --name neo4j-kmc \
36         -p 7474:7474 -p 7687:7687 \
37         -e NEO4J_AUTH=neo4j/password \
38         -e NEO4J_PLUGINS='["apoc"]' \
39         --restart always \
40         neo4j:latest
41     echo "🔍 Neo4j started. Access it at http://localhost:7474"
42 else
43     echo "🔍 Neo4j container 'neo4j-kmc' already exists."
44     if [ "$($DOCKER_CMD inspect -f '{{.State.Running}}' neo4j-kmc)" != "true" ]; then
45         echo "🔍 Starting Neo4j container..."
46         $DOCKER_CMD start neo4j-kmc
47     fi
48 fi
49
50 # 4. Check for Ollama
51 if ! command -v ollama &> /dev/null; then
52     echo "🔍 Ollama not found in PATH. Please install it from https://ollama.com"
53 else
54     if ollama list | grep -q "qwen3.6:35b"; then
55         echo "🔍 Model qwen3.6:35b already exists in Ollama."
56     else
57         echo "🔍 Pulling Qwen model (qwen3.6:35b)..."

```

```

58         ollama pull qwen3.6:35b
59     fi
60 fi
61
62 # 5. Create .env template if it doesn't exist
63 if [ ! -f .env ]; then
64     echo "❏ Creating .env template..."
65     echo "NEO4J_URI=bolt://localhost:7687" > .env
66     echo "NEO4J_USER=neo4j" >> .env
67     echo "NEO4J_PASSWORD=password" >> .env
68 fi
69
70 echo "❏ Setup complete!"
71 echo "❏ You can now run the pipeline: 'python kg_generator.py'"

```

8 Setup Runner

Purpose: A Python utility script to launch the `setup.sh` installation routine in a detached background process.

Listing 8: `run_setup.py`

```

1  import subprocess
2  import os
3  import sys
4
5  def run_setup_background():
6      """
7      Runs setup.sh in the background and redirects all output to setup.log.
8      Designed for execution on an Ubuntu server.
9      """
10     script_name = "setup.sh"
11     log_name = "setup.log"
12
13     # Check if setup.sh exists
14     if not os.path.exists(script_name):
15         print(f"❏ Error: {script_name} not found in the current directory.")
16         return
17
18     # Ensure the script is executable
19     os.chmod(script_name, 0o755)
20
21     print(f"❏ Launching {script_name} in the background...")
22     print(f"❏ Output will be logged to {log_name}")
23
24     try:
25         # Open the log file for writing
26         with open(log_name, "w") as log_file:
27             # subprocess.Popen starts the process without waiting for it to finish
28             # We use 'bash' explicitly to ensure it runs correctly on Ubuntu
29             process = subprocess.Popen(
30                 ["bash", script_name],
31                 stdout=log_file,
32                 stderr=subprocess.STDOUT, # Redirect stderr to the same log file
33                 start_new_session=True    # Detach from the current terminal session

```

```

34         )
35
36         print(f"🔧 Setup background process started successfully!")
37         print(f"🔧 PID: {process.pid}")
38         print(f"🔧 You can monitor progress with: tail -f {log_name}")
39
40     except Exception as e:
41         print(f"🔧 Failed to start the setup process: {e}")
42
43 if __name__ == "__main__":
44     run_setup_background()

```

9 Model Loader (GPU 0)

Purpose: A sophisticated bash script to trigger the loading of the Qwen model into GPU 0 memory in the background and verify its active status via polling.

Listing 9: load_model_gpu_0.sh

```

1  #!/bin/bash
2
3  # =====
4  # Ollama Model GPU Loader & Verification Script (GPU 0)
5  # =====
6  # Description: This script triggers the loading of an Ollama model into GPU 0
7  #               memory in the background and verifies the status.
8  # =====
9
10 # --- Configuration ---
11 MODEL_NAME="qwen3.6:35b"
12 LOG_FILE="ollama_gpu_0_load.txt"
13 GPU_ID=0 # The index of the GPU to use
14 OLLAMA_PORT=11434
15
16 # ANSI Color Codes for Premium UI
17 GREEN='\033[0;32m'
18 BLUE='\033[0;34m'
19 CYAN='\033[0;36m'
20 YELLOW='\033[1;33m'
21 RED='\033[0;31m'
22 BOLD='\033[1m'
23 NC='\033[0m' # No Color
24
25 # Header
26 echo -e "${CYAN}${BOLD}===== ${NC}"
27 echo -e "${CYAN}${BOLD}          OLLAMA GPU 0 LOAD & TEST UTILITY          ${NC}"
28 echo -e "${CYAN}${BOLD}===== ${NC}"
29
30 # Initialize log file
31 echo "--- Ollama GPU 0 Load Session: $(date) ---" > "$LOG_FILE"
32
33 # --- Function: Ensure Server is Running ---
34 ensure_server_running() {
35     echo -e "${YELLOW}[*] Action: Checking Ollama server status on port $OLLAMA_PORT
...${NC}"

```

```

36
37     if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
38         echo -e "${GREEN}[+] Ollama server is already active on port $OLLAMA_PORT.${NC}
39     }"
40     return 0
41 fi
42
43 echo -e "${YELLOW}[!] Ollama server not detected on port $OLLAMA_PORT. Starting
44 isolated instance...${NC}"
45
46 # CRITICAL: Set CUDA_VISIBLE_DEVICES for the SERVER process
47 export CUDA_VISIBLE_DEVICES=$GPU_ID
48 export OLLAMA_HOST="127.0.0.1:$OLLAMA_PORT"
49
50 # Start server in background
51 nohup ollama serve >> "$LOG_FILE" 2>&1 &
52
53 # Wait and verify status
54 echo -n "Waiting for API availability"
55 for i in {1..20}; do
56     echo -n "."
57     if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
58         echo -e "\n${GREEN}[SUCCESS] Ollama server (GPU $GPU_ID) is up!${NC}" |
59         tee -a "$LOG_FILE"
60         return 0
61     fi
62     sleep 2
63 done
64
65 echo -e "\n${RED}[ERROR] Server failed to start within 40 seconds.${NC}" | tee -a
66 "$LOG_FILE"
67 exit 1
68 }
69
70 # --- Function: Load Model ---
71 load_to_gpu_bg() {
72     echo -e "${YELLOW}[*] Action: Triggering model load for '$MODEL_NAME' on port
73     $OLLAMA_PORT...${NC}"
74
75     # Trigger a run with empty input to force the model into VRAM
76     export OLLAMA_HOST="127.0.0.1:$OLLAMA_PORT"
77     nohup ollama run $MODEL_NAME "" >> "$LOG_FILE" 2>&1 &
78
79     BG_PID=$!
80     echo -e "${GREEN}[+] Load command dispatched (PID: $BG_PID).${NC}" | tee -a "
81     $LOG_FILE"
82 }
83
84 # --- Function: Verification Test ---
85 run_verification_test() {
86     echo -e "\n${YELLOW}[*] Action: Verifying GPU VRAM status...${NC}"
87     echo -n "Waiting for model initialization"
88
89     MAX_ATTEMPTS=12
90     SUCCESS=false
91
92     for ((i=1; i<=MAX_ATTEMPTS; i++)); do
93         echo -n "."
94         if OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps 2>/dev/null | grep -q "

```

```

89     $MODEL_NAME"; then
90         echo -e "\n${GREEN}${BOLD}[SUCCESS] Model '$MODEL_NAME' is active in GPU
memory!${NC}" | tee -a "$LOG_FILE"
91         SUCCESS=true
92         break
93     fi
94     sleep 5
95 done
96
97 if [ "$SUCCESS" = true ]; then
98     echo -e "${BLUE}-----${NC}"
99     echo -e "${BOLD}Current GPU Load Status (Port $OLLAMA_PORT):${NC}"
100    OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps | grep -E "NAME|${MODEL_NAME}"
101    echo -e "${BLUE}-----${NC}"
102    return 0
103 else
104     echo -e "\n${RED}${BOLD}[FAILURE] Model verification timed out.${NC}" | tee -a
"$LOG_FILE"
105     return 1
106 fi
107 }
108 # --- Main Execution ---
109 ensure_server_running
110 load_to_gpu_bg
111 run_verification_test
112
113 echo -e "\n${CYAN}Done.${NC}"

```

10 Model Loader (GPU 1)

Purpose: A sophisticated bash script to trigger the loading of the Qwen model into GPU 1 memory in the background and verify its active status via polling.

Listing 10: load_model_gpu_1.sh

```

1  #!/bin/bash
2
3  # =====
4  # Ollama Model GPU Loader & Verification Script (GPU 1)
5  # =====
6  # Description: This script triggers the loading of an Ollama model into GPU 1
7  #               memory in the background and verifies the status.
8  # =====
9
10 # --- Configuration ---
11 MODEL_NAME="qwen3.6:35b"
12 LOG_FILE="ollama_gpu_1_load.txt"
13 GPU_ID=1 # The index of the GPU to use
14 OLLAMA_PORT=11435
15
16 # ANSI Color Codes for Premium UI
17 GREEN='\033[0;32m'
18 BLUE='\033[0;34m'
19 CYAN='\033[0;36m'

```

```

20 YELLOW='\033[1;33m'
21 RED='\033[0;31m'
22 BOLD='\033[1m'
23 NC='\033[0m' # No Color
24
25 # Header
26 echo -e "${CYAN}${BOLD}===== ${NC}"
27 echo -e "${CYAN}${BOLD}          OLLAMA GPU 1 LOAD & TEST UTILITY          ${NC}"
28 echo -e "${CYAN}${BOLD}===== ${NC}"
29
30 # Initialize log file
31 echo "--- Ollama GPU 1 Load Session: $(date) ---" > "$LOG_FILE"
32
33 # --- Function: Ensure Server is Running ---
34 ensure_server_running() {
35     echo -e "${YELLOW}[*] Action: Checking Ollama server status on port $OLLAMA_PORT
...${NC}"
36
37     if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
38         echo -e "${GREEN}[+] Ollama server is already active on port $OLLAMA_PORT. ${NC}"
39     fi
40     return 0
41 fi
42
43 echo -e "${YELLOW}[!] Ollama server not detected on port $OLLAMA_PORT. Starting
isolated instance...${NC}"
44
45 # CRITICAL: Set CUDA_VISIBLE_DEVICES for the SERVER process
46 export CUDA_VISIBLE_DEVICES=$GPU_ID
47 export OLLAMA_HOST="127.0.0.1:$OLLAMA_PORT"
48
49 # Start server in background
50 nohup ollama serve >> "$LOG_FILE" 2>&1 &
51
52 # Wait and verify status
53 echo -n "Waiting for API availability"
54 for i in {1..20}; do
55     echo -n "."
56     if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
57         echo -e "\n${GREEN}[SUCCESS] Ollama server (GPU $GPU_ID) is up! ${NC}" |
tee -a "$LOG_FILE"
58         return 0
59     fi
60     sleep 2
61 done
62
63 echo -e "\n${RED}[ERROR] Server failed to start within 40 seconds. ${NC}" | tee -a
"$LOG_FILE"
64 exit 1
65 }
66
67 # --- Function: Load Model ---
68 load_to_gpu_bg() {
69     echo -e "${YELLOW}[*] Action: Triggering model load for '$MODEL_NAME' on port
$OLLAMA_PORT...${NC}"
70
71     # Trigger a run with empty input to force the model into VRAM
72     export OLLAMA_HOST="127.0.0.1:$OLLAMA_PORT"
73     nohup ollama run $MODEL_NAME "" >> "$LOG_FILE" 2>&1 &

```

```

73
74     BG_PID=$!
75     echo -e "\n${GREEN}[+] Load command dispatched (PID: $BG_PID).${NC}" | tee -a "
76     $LOG_FILE"
77 }
78 # --- Function: Verification Test ---
79 run_verification_test() {
80     echo -e "\n${YELLOW}[*] Action: Verifying GPU VRAM status...${NC}"
81     echo -n "Waiting for model initialization"
82
83     MAX_ATTEMPTS=12
84     SUCCESS=false
85
86     for ((i=1; i<=MAX_ATTEMPTS; i++)); do
87         echo -n "."
88         if OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps 2>/dev/null | grep -q "
89 $MODEL_NAME"; then
90             echo -e "\n${GREEN}${BOLD}[SUCCESS] Model '$MODEL_NAME' is active in GPU
91 memory!${NC}" | tee -a "$LOG_FILE"
92             SUCCESS=true
93             break
94         fi
95         sleep 5
96     done
97
98     if [ "$SUCCESS" = true ]; then
99         echo -e "${BLUE}-----${NC}"
100        echo -e "${BOLD}Current GPU Load Status (Port $OLLAMA_PORT):${NC}"
101        OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps | grep -E "NAME|${MODEL_NAME}"
102        echo -e "${BLUE}-----${NC}"
103        return 0
104    else
105        echo -e "\n${RED}${BOLD}[FAILURE] Model verification timed out.${NC}" | tee -a
106        "$LOG_FILE"
107        return 1
108    fi
109 }
110 # --- Main Execution ---
111 ensure_server_running
112 load_to_gpu_bg
113 run_verification_test
114 echo -e "\n${CYAN}Done.${NC}"

```

11 Requirements

Purpose: Defines the required Python packages for running the dual-GPU knowledge graph pipeline.

Listing 11: requirements.txt

```

1 neo4j
2 ollama

```

```
3 python-dotenv
4 tqdm
```